For any suggestions or feedback regarding these notes,
please contact [Pragy Agarwal]

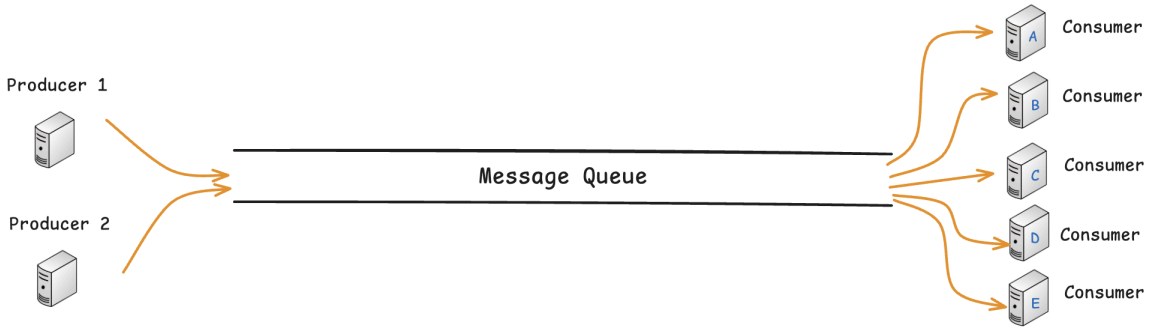
Messaging Queues

Asynchronous communication mechanism where **messages** (data, events) are placed onto a **queue** by **producers** (senders) and later processed or consumed by **consumers** (receivers).

Key Idea

Pub-Sub architecture pattern: Decouple sending and receiving of data.
Senders can push messages without waiting for the receivers to process them, enabling better scalability and fault-tolerance.

(homework: find the differences b/w PubSub, Observer, Event Handler patterns)



Producers (aka Publishers/Senders)

Push the messages into the queue

Consumers (aka Subscribers/Receivers)

Consume the messages from the queue, processes them, then acknowledges the consumption

Queue

Holds the messages in FIFO order
(more on the ordering later)

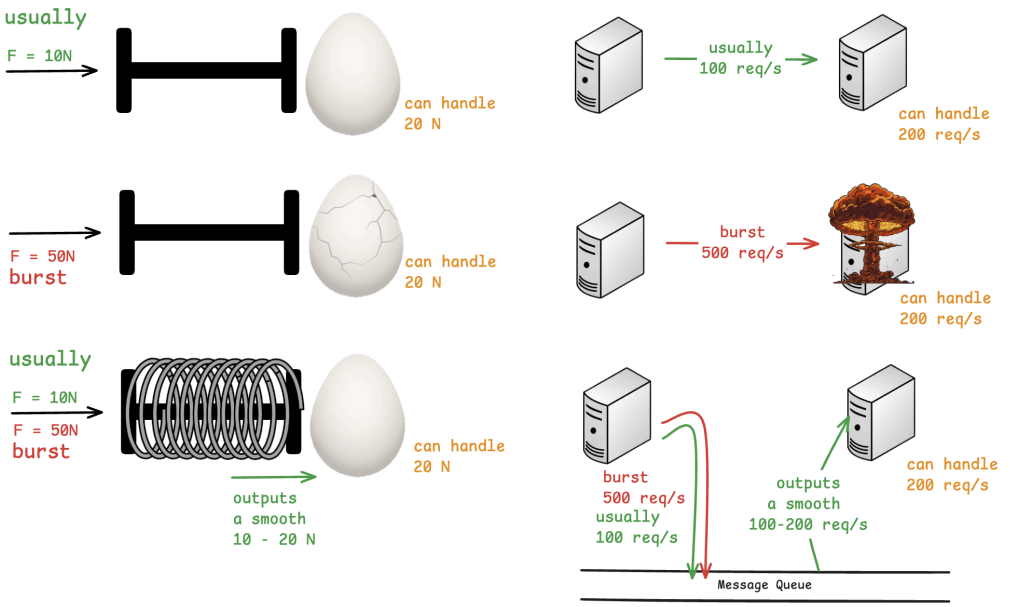
Benefits of Pub-Sub

1. **Decoupling**

    - **Scalability:** Can scale the producers & consumers independently
      eg. in a Video processing service, producers will insert videos to be processed, consumers will consume & process the videos.
      We will need many more consumers than producers.

    - **Fault Tolerance:** If producers fail, the consumers can continue processing the events present in the queue. If consumers fail, the producers can continue producing without interruption

2. **Load Leveling (Shock Absorber):** Consumers are kept safe from sudden traffic bursts



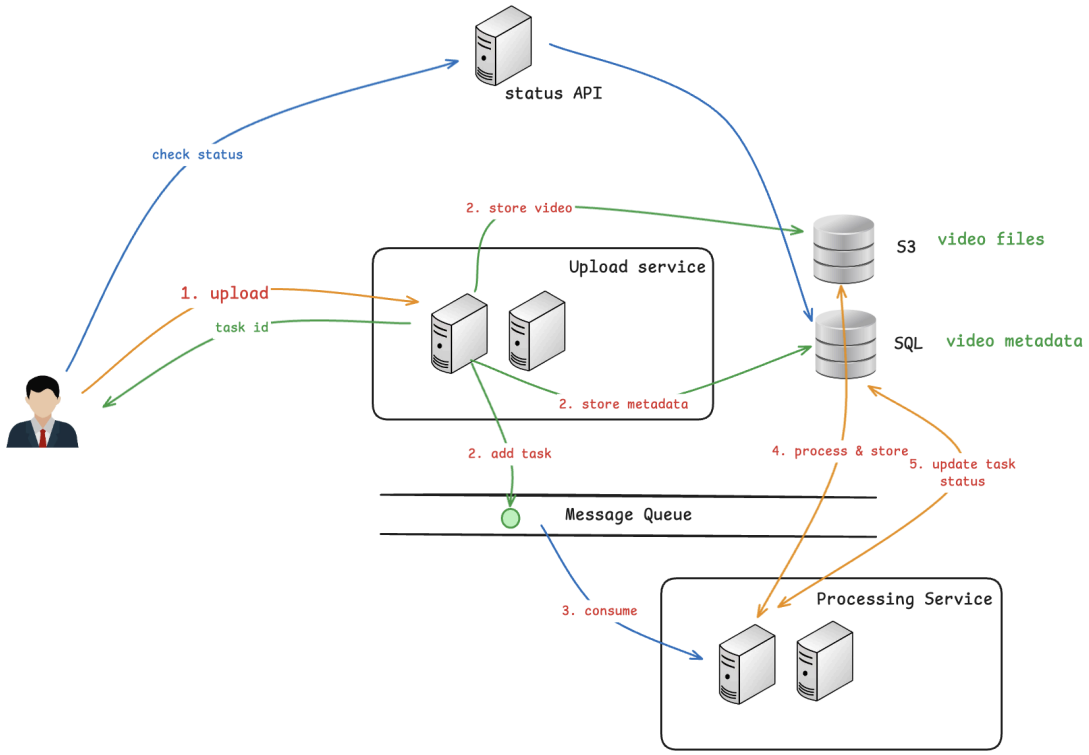
    - *Helps prevent cascading failure.*

3. **Buffering & Reliability:** Messages are persisted in the queue, ensuring that they're not lost even if a consumer crashes
   (note: this is about the messages (requests) not being lost, not about the service not being down)

Usecases

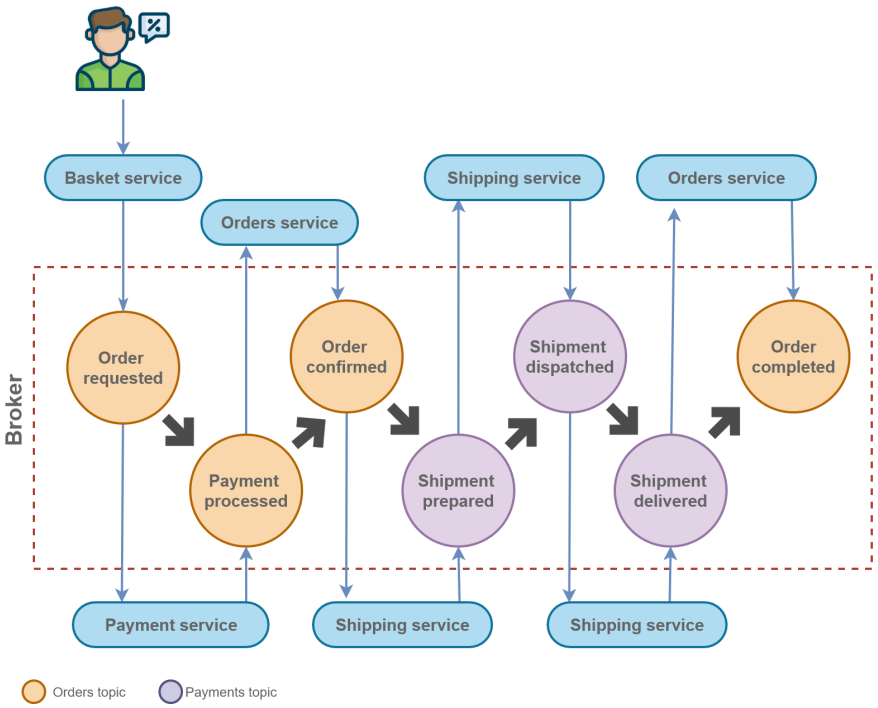
1. **Communication b/w microservices:** instead of making direct API calls (REST/SOAP/RPC), services can communicate by message passing

2. **Async Processing / Task Scheduling:** for example, a Video processing service

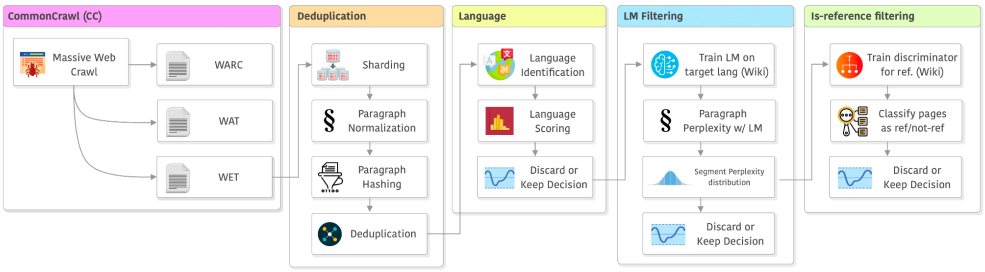


3. **Event-Driven Pipelines / Event tracking:**
    a. User Action analytics: clicks / page views / likes / video views
    b. Fraud Detection / Attack Detection: detect when a malicious activity is happening and trigger corrective measures

c. Anomaly Detection / Alerts: observability *(more on this later)*



4. **Data-Driven Pipelines:**
   a. Web crawling pipelines
   b. Machine Learning pipelines
   c. Data Analytics



5. **Logging:** and log aggregation to a central repository

Q: Is a messaging queue always asynchronous?

Yes, by design!
Producers just push messages without worrying about when or how the messages are consumed.
The consumers do not (can not) block the producers, and vice versa.

Cons of using Message Queues

1. Not possible to do synchronous tasks
2. Higher latency compared to sync API calls: latency here means the amount of time between adding task to the queue, and the task actually getting processed by some consumer.
   (although usually < 2ms, if the queue is empty)
3. Needs additional infrastructure
   - A messaging queue will be a cluster of servers.
4. Higher network overhead

Need to justify

Don't use Message Queues "just because". They're complex to setup, use, & maintain.
So if you use a message Queue, you should be able to justify the need for it.

Most Common Message Queues

1. **Apache Kafka:** the most popular message queue.
2. **Rabbit MQ:** also common, but not as powerful/featureful as Kafka
3. **Amazon SQS:** managed service (you don't have to setup anything)

Apache Kafka

https://kafka.apache.org/

1. **Open Source:** built at Linkedin, now part of Apache
2. **High Throughput:** a cluster can handle up to `10 million msg/s` (trillion/day)
3. **Ultra Low Latency:** claims `< 2ms` latency from message insertion to consumption (provided that queue is empty)
   - latency overhead of Kafka itself is very low
4. **Horizontal Scalability:** automatically manages partitions across multiple servers (brokers)
   `1000s` of brokers (servers), `100,000s` of partitions, `petabytes` of data
5. **Fault Tolerant:** automatically replicates messages across multiple servers, and it stores the messages on disk
6. **High Availability:** can deploy clusters across availability zones
7. **Persistent:** saves the messages on disk to ensure they're not lost. Multiple consumer groups can consume the same message.
8. **Message Ordering:** provides strict* message ordering *(*within each partition only)*

9. **Battles tested:** used by *all* large companies, and tens of thousands of other companies

Message (record/event/task)

An event records the fact that "something happened" in the world or in your business.

```
{
    [key]          // for message partitioning within topics
    value          // actual payload being transmitted (json/string/binary)
    timestamp      // creation time or log append time
    [headers]      // other data for routing/processing
    topic          // decides the "queue" in which this message belongs
}
```

https://kafka.apache.org/39/javadoc/org/apache/kafka/clients/producer/ProducerRecord.html

Note:

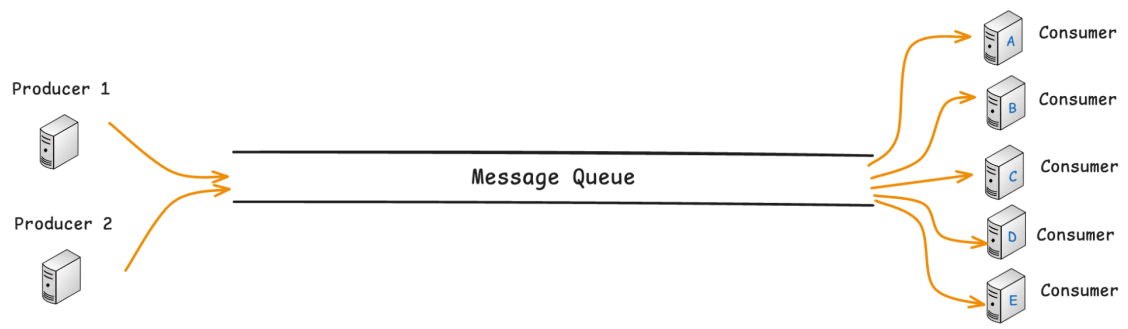
- Kafka is **append only**
  - can insert messages
  - can read messages in a *strict FIFO order (like a queue)
    **ordering is only within a partition*
  - can not update/delete messages
    - automatically deletes messages after 7 days
  - cannot search messages

Message size should be limited to a few Kbs at max.

Producers & Consumers

Producers are applications that publish (write) events to Kafka.
Consumers subscribe to (read and process) these events.

Producers and consumers are fully decoupled and agnostic of each other — producers never need to wait for consumers.



Is Kafka push or pull?

Producers push messages into Kafka. Consumers pull messages from Kafka. [ref]

In some Messaging queues like RabbitMQ, the messages are pushed to the consumers. Kafka doesn't do that.

Consumer Groups (scalability)

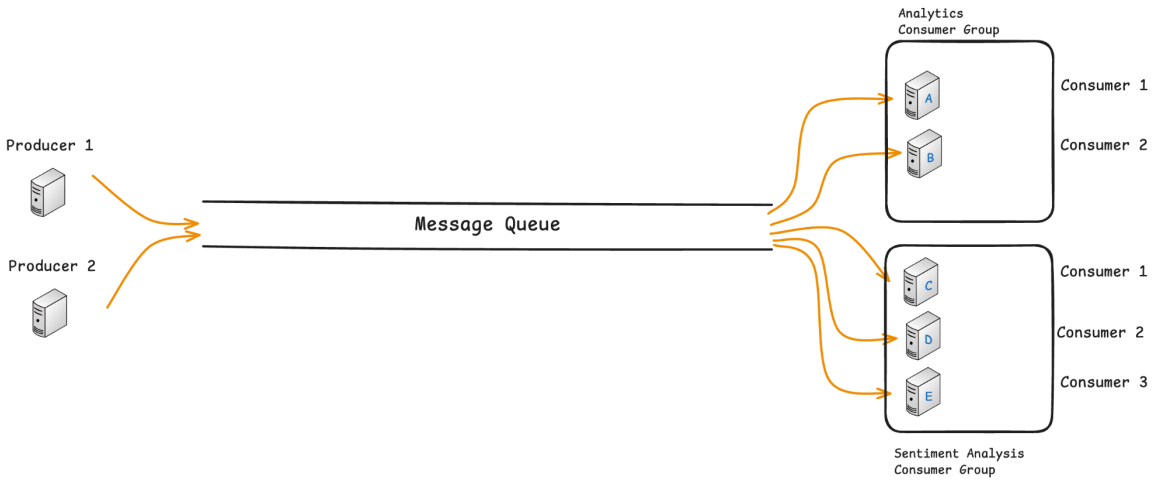
Challenge

A single consumer might not be able to process all the messages (10M/s).

Solution

So, we need multiple consumers for each task category.

- Consumers must provide a `group.id` when they join.
- Kafka assigns a special "coordinator" to manage each consumer group
    - consumers send heartbeats to the coordinator to remain alive



Q: Can a consumer be part of multiple consumer groups?

No. Every consumer belongs to exactly 1 consumer group.

Notice that a single server can run multiple consumers, which can belong to different consumer groups (not recommended, but possible).
Each consumer is basically a process. And each consumer process will belong to exactly 1 consumer group.

Topics

Challenge

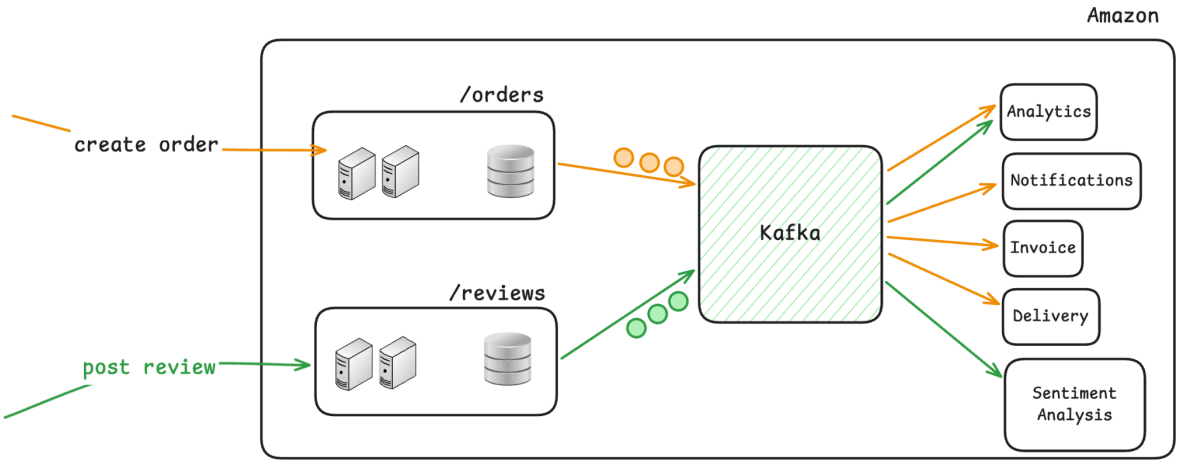
Every consumer has to process all the messages.
Not every consumer cares about every message. We want the consumers to be able to selectively process the messages.

Example

Let's consider the use case of an e-commerce website like Amazon.
Suppose a user can perform two actions

1. Place an order
2. Post a review

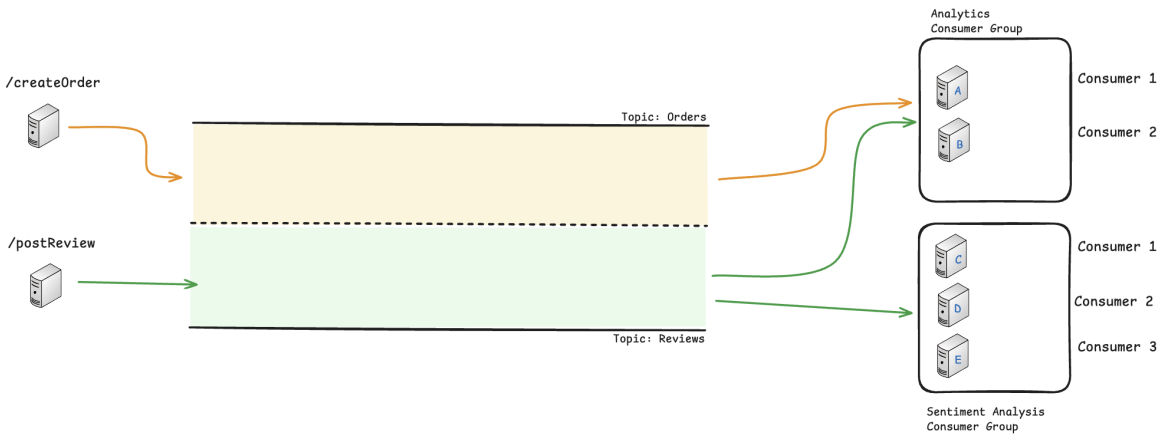
Suppose there are multiple services that need to consume various events



Different services (consumer groups) want to consume different types of events.
At present, all services must consume all messages, which leads to a lot of wastage of resources.

Solution

Categorize the messages into various topics. Each message belongs to exactly 1 topic.
Think of a topic as a folder, and the messages as files. *(Kafka literally stores then like this!)*



Note: when producers insert messages into Kafka, they MUST provide the topic.
https://kafka.apache.org/39/javadoc/org/apache/kafka/clients/producer/ProducerRecord.html

> *Topics in Kafka are always multi-producer and multi-subscriber: a topic can have zero, one, or many producers that write events to it, as well as zero, one, or many consumers that subscribe to these events.*
> *Events in a topic can be read as often as needed—unlike traditional messaging systems, events are not deleted after consumption. Instead, you define for how long Kafka should retain your events through a per-topic configuration setting, after which old events will be discarded. Kafka's performance is effectively constant with respect to data size, so storing data for a long time is perfectly fine. [ref]*

Partitions (message ordering & scalability++)

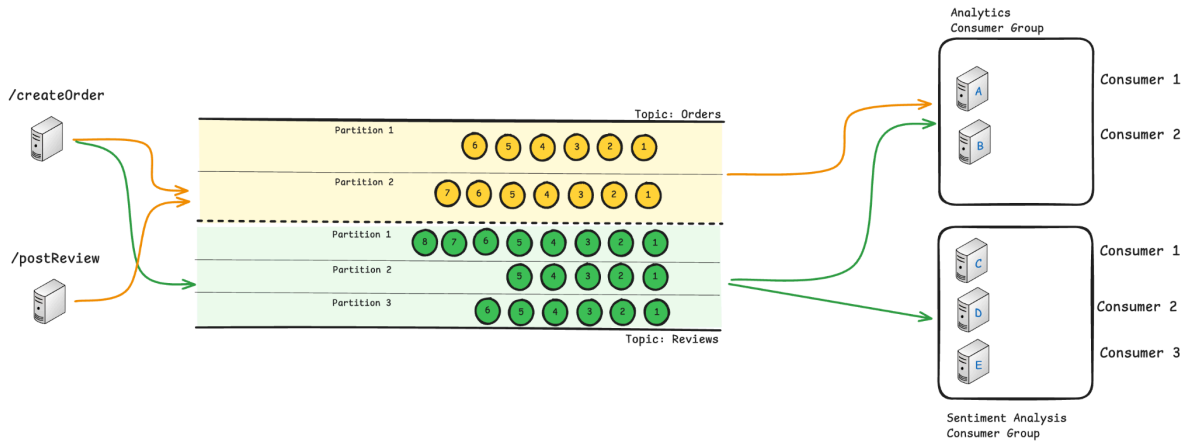
Challenge

1. A single server is not enough to store all messages in a topic
   for example, a topic like "1-1 chat messages" or “tweets” might contain billions of events
   - so we need to “partition” the topic further so that we can store it across multiple servers (brokers)

2. Sometimes, we need to provide strict message ordering for the events that are being consumed (for example, we want strict ordering for chat messages, or for user GPS coordinate changes)

Solution

Since a topic is too large to store in a single server, we can partition the topic across multiple servers.
This will allow our events to be stored across multiple servers, and also, our publishers & subscribers can write-to/read-from multiple servers, achieving higher throughput.



Queues are FIFO - they provide strict message ordering.
As long as the topic was contained within a single server, FIFO ordering was easy.

It is extremely challenging to provide strict message ordering across multiple servers *(because you will need distributed locks - complex, slow)*

Each partition is stored completely within a single Kafka Broker (server). (of course, each partition will be replicated across multiplied brokers)

Therefore, Kafka only guarantees message ordering within each partition (not across partitions)

Whenever an event is inserted into Kafka, it is inserted into 1 of the partitions of the topic.

Which partition is an event inserted in?

If the event has a `key` attribute, then the `hash(key) % num_partitions` is used to decide the partition. Otherwise, it goes to a random partition of the topic. [ref]

All messages that belong to the same topic, and have the same key will go into the same partition.

For example, if we want to maintain strict ordering for *1-1 chat messages*, our chat events would look like

```
{
    key: conversation_id
    value: {
        message: "Hi",
        prev_message_id: "...",
        ...
        attachment_url: ...,
        geolocation: ...,
```

```
        },
        timestamp: ...
        [headers]: ...
    }
```

Because all the messages of a given conversation (Upinta ←→ Subhadeep) will have the same conversation id, they will be inserted into the same partition. Which means that the message ordering will be maintained within the conversation.

Kafka guarantees strict ordering in each partition: *any consumer of a given topic-partition will always read that partition's events in exactly the same order as they were written.*

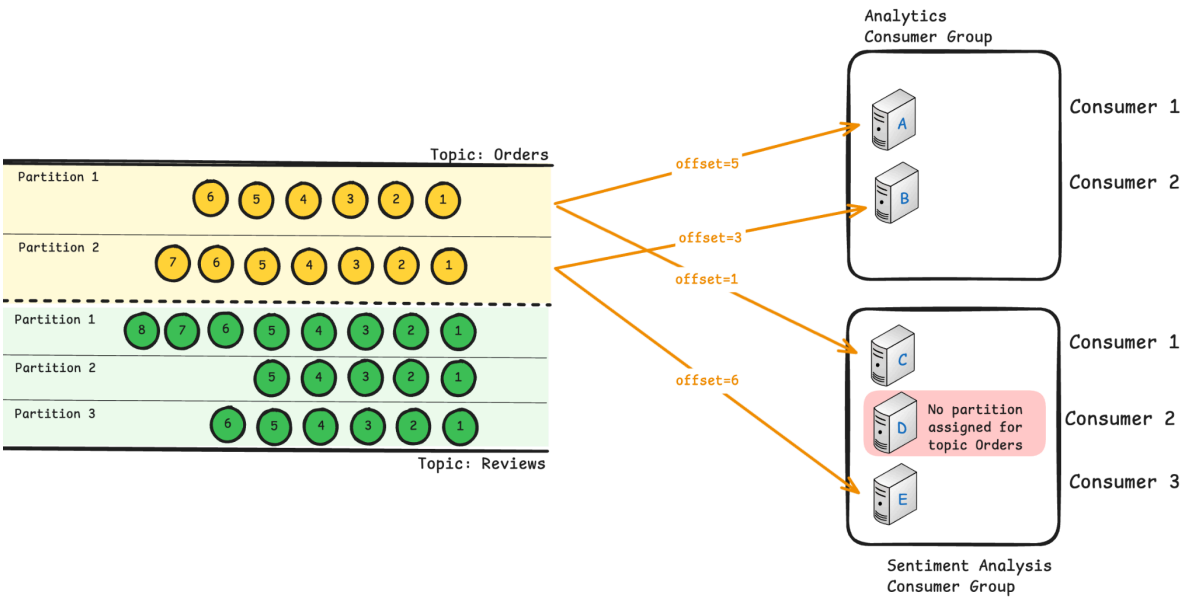
Consumer-Partition Mapping & Consumer Offsets

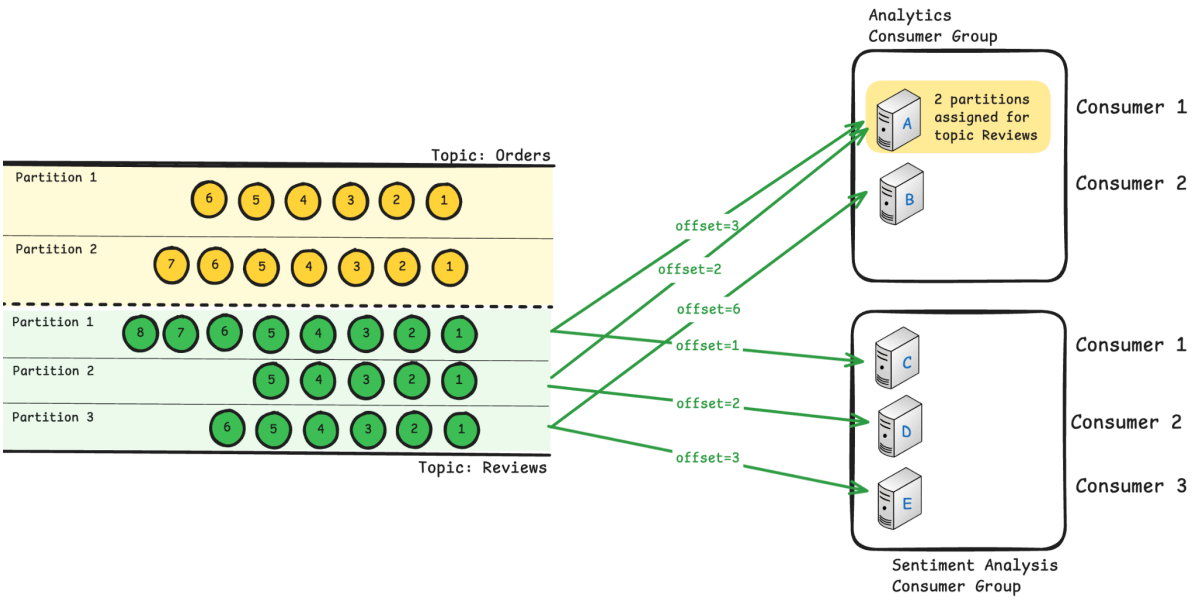
Each partition of a topic is consumed by exactly 1 consumer within each consumer group.
For every such (partition, consumer) pair, Kafka maintains an integer `offset` that specifies which message in the partition should the consumer read next.
Consumer offsets are maintained in a special topic named `__consumer_offsets`

This mapping b/w consumers & partitions is maintained by the coordinator (a Kafka server which is managing the consumer group)

- **Can the same partition be assigned to multiple consumers?**
    - Multiple consumers within the same consumer group? No
    - Multiple consumers, each in a different consumer group? Yes
- **Can multiple partitions be assigned to the same consumer?**
    - Yes

Each partition of a topic is consumed by exactly 1 consumer within each group that subscribes to that topic.





Therefore, we must always ensure that `#partitions >= #consumers`.

1 partition is assigned to exactly 1 consumer. Same consumer can get multiple partitions. But same partition cannot go to multiple consumers.

If you have more consumers than partitions, some consumers will remain idle!
Since we might add more consumers later on (to scale out), we always create more partitions than needed preemptively.
Note that

- number of partitions is configured per topic
- when a topic is created, the number of partitions must be decided beforehand
    - it is difficult to change the number of partitions in a topic later on
        - *possible, but requires downtime (because you will have to reshuffle the messages across the servers)*
        - *& can break the delivery guarantees (because the offset will change)*

- **Rebalancing:** If a consumer joins or leaves, Kafka redistributes the partitions among the group's consumers. (group's coordinator maintains a heartbeat with the consumers of the group.)

For example, if we're creating a new Kafka topic called "ImageUploads", then we will try to guess the max number of consumers we might need in the future that consume from this topic, and set the number of partitions higher than that.

Note: if you create a topic with 5000 partitions, that doesn't mean that you need 5000 Kafka servers. A single Kafka server (Broker) can hold multiple partitions. But each partition lives fully inside a single Broker (although it is replicated to multiple servers)
Kafka follows partition based sharding, and has master-slave replication

Delivery Guarantees in Kafka

- **At most once:** Messages may be lost but are never redelivered.
  either deliver 0 times (lost), or 1 time. But not more than once (no duplicate deliveries).

- **At least once:** Messages are never lost but may be redelivered.
  either delivered 1 time, or 2+ times (duplicates)

- **Exactly once:** each message is delivered once and only once. *(this is what people actually want)*

1. **Producer:**

What happens if the producer/kafka-broker crashes during insertion?

Kafka broker crash: When a message is published (inserted), it is "committed" to a log on the disk to ensure durability.

Producer crash: If the producer crashes while producing the message, it can just re-produce it when it restarts. *(retry mechanism)*

The producer API is (optionally) idempotent [ref], so the producer can just re-publish the message, and it will be committed to the log only once.

Note: you can turn off this idempotency if you want lower latencies - but this can cause duplicate messages.

So, thanks to automatic retries by producers, and idempotency of producer API, the producers are ***inserting "exactly once"***

2. **Consumer:** The consumer must "commit" its offset to Kafka
    a. **Read event => Commit offset => Process event:** this gives at-most once delivery (task might get lost) [ref]
    If a consumer reads a message, commits the offset, but crashes before it is able to process the message, then the message will never get processed by ***this*** consumer group (any replacement consumer will start consuming from the next offset)

    b. **Read event => Process event => Commit offset:** this give at-least once delivery [ref]
    If the consumer reads the message, processes it successfully, but before it is able to commit the offset the consumer crashes. When the replacement consumer picks up this partition, it will once again receive the same event. Duplicate processing is possible!

So, the delivery guarantees depend on how the consumer behaves.
*(**producer is guaranteed to be exactly once**. Consumer behaviour can change the guarantee to at-least/at-most/exactly once)*

So what delivery guarantee does Kafka really provide?

By default, Kafka guarantees **at-least-once** delivery (producers retry when publishing fails, consumers process before committing)
Therefore, by default, in Kafka, you can get multiple deliveries for the same event!

If you want at-most-once, you can disable retries on the producer (not necessary always, since the producer API is idempotent), and your consumers can commit the offset before processing the message.

Exactly once delivery is available using transactions, when you're consuming from one topic and inserting into another (Kafka Streams). If not using Streams, consumer must store the offset in the same place as the output data. *(basically left to you - the developer)*

Note: the delivery guarantees depend on your producers and consumers. A consumer CAN violate the offset if it wants to - for example, if it needs to re-process some data. [ref]

Brokers (sharding & replication)

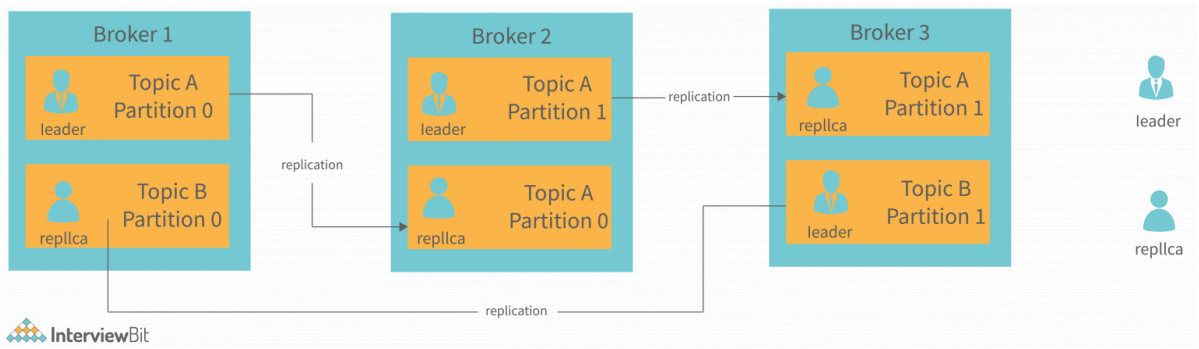
A Broker in Kafka is just a server on which Kafka is installed.
Brokers store the message in logs.

- **Sharding:** Messages in Kafka are sharded by topic & partition.
- **Replication:** Each topic's partitions are replicated across multiple brokers in a master-slave (quorum) configuration.

- Writes go to (at least) master, but a majority of replicas. Reads go to any majority of replicas (including master).
    - A message is considered "committed" only when it is written to all replicas (master + all slaves)
    - Consumers can only see committed messages [ref]
    - Producers have an option to wait for commit synchronously or write to master and let commit happen asynchronously [ref]

      Basically, producer can choose to get acknowledgement quickly after writing to only 1 replica (leader).
      But consumer will see the message only after it has been fully replicated across all replicas.

- Replication factor can be configured per topic.
- Masters are equally distributed across brokers
- Kafka prefers consistency over availability [ref]



How long does Kafka store a message?

By default for 7 days.

- during this time, any message may be consumed by any number of consumers (but only once per consumer group).
- after 7 days, Kafka deletes the message irrespective of whether it has been consumed or not.

This deletion is configurable by either time, or partition size

Zookeeper (configuration)

1. Which servers (brokers) are up and running
    a. heartbeat / health-check on them
2. Which topics exist
3. How many partitions does each topic have
4. Which partition maps to which consumer in a consumer group
5. Master-Slave configuration data for each partition

6. Other: ACL, quotas, internal data, configuration, ..

Zookeeper vs KRaft

Zookeeper has limitations, due to which Kafka could not handle more than 200,000 partitions in a cluster. [ref]

Starting from Kafka 2.8, support for KRaft was added. Deprecated Zookeeper from Kafka 3.5 Starting from Kafka 4.0 (not released yet, as of March 4 2025) Zookeeper will be completely removed.

Raft Consensus Algorithm: https://raft.github.io/

Zookeeper

https://zookeeper.apache.org/

ZooKeeper is a distributed coordination service.
It exposes a simple set of primitives that distributed applications can build upon to implement higher level services for synchronization, configuration maintenance, and groups and naming.

Zookeeper provides immediate consistency.

Zookeeper Filesystem

Zookeeper basically exposes a special filesystem over the network via various APIs.

- Directories & Files are called ZNodes
- ZNodes can be persistent or ephemeral
    1. **Persistent:** normal files & directories
    2. **Ephemeral:** files/directories "owned" by a server (zookeeper maintains a persistent session with the owning server)
        - when the server disconnects or loses heartbeats, this ephemeral ZNode is deleted by Zookeeper
        - *Basically, the file will remain alive as long as the server owning it remains alive. When owning server crashes, the file also get deleted.*
        - there's exactly 1 owner
        - only the owner can update/delete the node. Other servers can read (watch), but not modify

You can store anything in zookeeper
- typically small files (few Kbs)
- typically store configuration data of a distributed system

```
/
 |-- db_secret_key
 |-- openai_api_key
 |-- aws_iam_key
 |-- master_slave_config
      | -- master_ip
      | -- slaves
            | -- a3b2c_ip
            | -- 2ccfd_ip
            | -- 13aa4_ip
            | -- dd12f_ip
```

Clients (other servers) can set a "watch" on any node. If the node changes, zookeeper will push the new data to the client via a persistent session.

```
string open_ai_api_key = ""

void fetch_openai_key():
    zookeeperClient.setWatch(
        "/openai_api_key",
        onKeyUdate
    )

void onKeyUpdate(value):
    open_ai_api_key = value

onServerStart => fetch_openai_key()
```

Replication

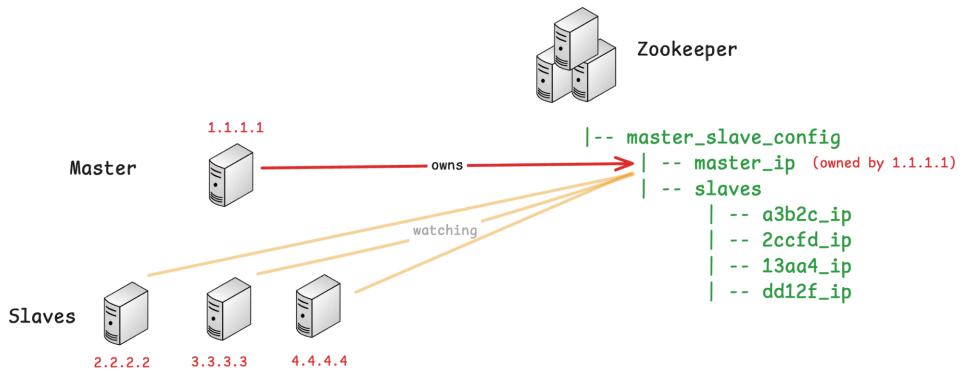
Zookeeper is itself replicated in a master-slave (quorum) configuration.
Zookeeper uses custom algos to maintain its master-slave config: FastLeaderElection.java

Managing Master-Slave configuration via Zookeeper

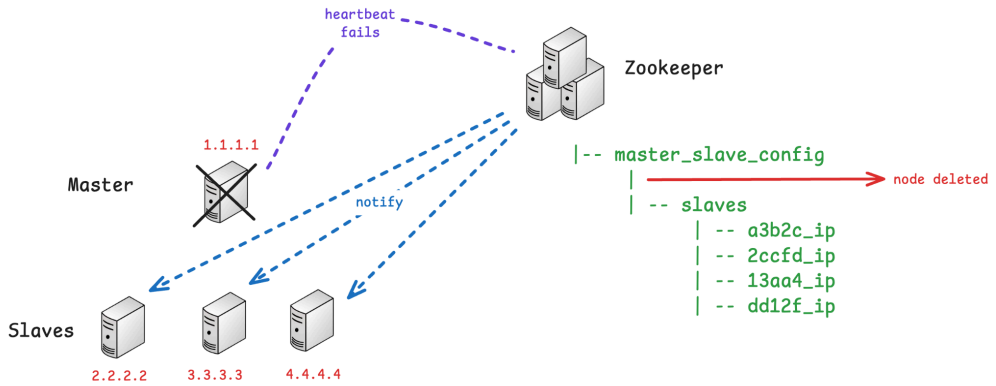
We can use Zookeeper to maintain our own Master-Slave configuration

1. Maintain master-slave config in Zookeeper
2. Master must own `master_ip` Ephemeral node

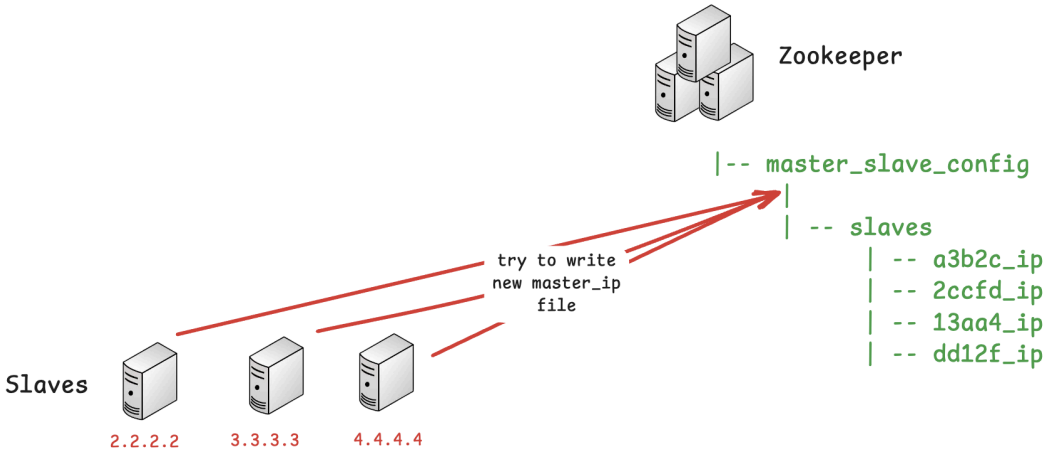
3. All slaves set a watch on `master_ip`



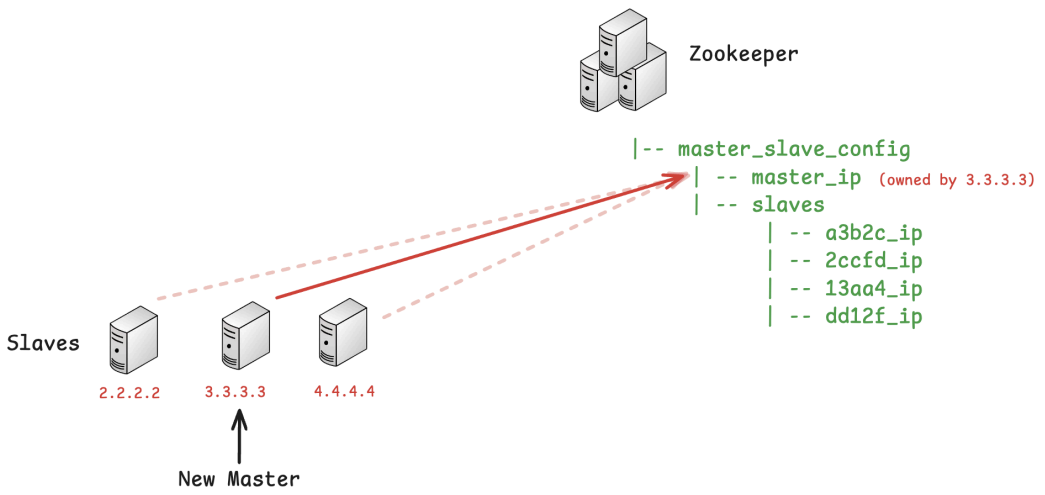
4. When master crashes
    a. Heartbeats to Zookeeper stop
    b. Zookeeper will delete the `master_ip` ephemeral node
    c. Zookeeper will notify all slaves



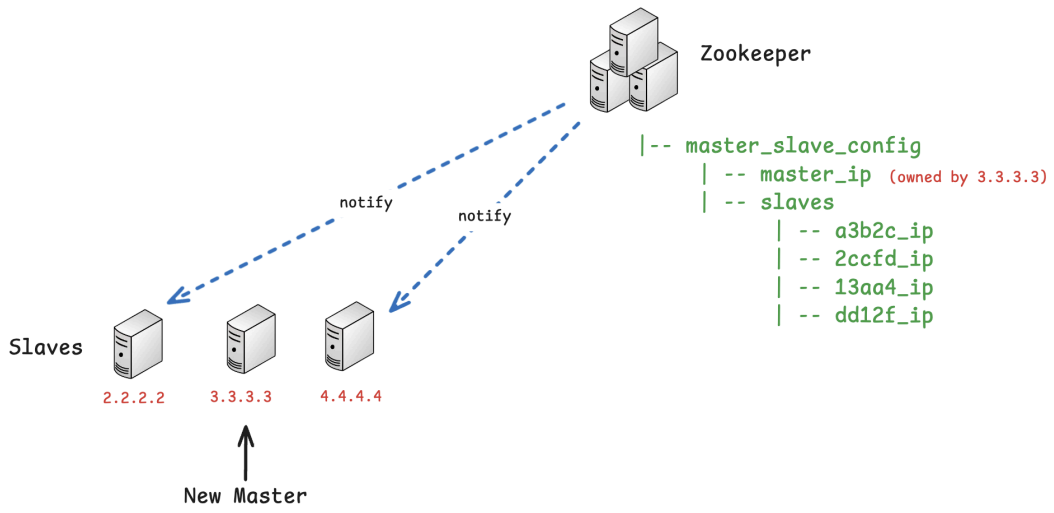
5. All slaves now try to create the `master_ip` file & put their own ip address inside it to become the owner (and hence, the master) themselves.



6. First slave to write will become the owner & the master. Zookeeper will prevent any other subsequent writes (only owner can modify an ephemeral node)



7. Zookeeper will notify all slaves about the new `master_ip`



More optimized approach:
https://zookeeper.apache.org/doc/current/recipes.html#sc_leaderElection

What happens if the old master comes back online?

It will join as a slave

How do we ensure that the new master has up-to-date data?

Need custom logic for that.

- **Sample Approach:** https://kafka.apache.org/documentation/#design:~:text=But%20followers%20themselves,these%20for%20now.
- **What Kafka does:** https://kafka.apache.org/documentation/#design:~:text=Kafka%20takes%20a,losing%20committed%20messages.

What happens if the Zookeeper master fails at the same time when our Master-Slave system's master fails?

Our master-slave system's master re-election will have to wait until the Zookeeper's master comes back online.

Resources (optional)

Kafka

1. **Basic Terminology:** https://www.conduktor.io/kafka/kafka-fundamentals/
2. **Motivation & Opiniated Design:** https://kafka.apache.org/documentation/#design
3. **Choosing the number of partitions:**
    - Apache Kafka Supports 200K Partitions Per Cluster
    - How to Choose the Number of Topics/Partitions in a Kafka Cluster? | Confluent
4. **Replication:**Kafka Topics Choosing the Replication Factor and Partition Count
5. **Offset management**
    - Kafka Consumer for Confluent Platform
    - Kafka - When to commit? - Quarkus
6. **Message Retention & Deletion**
    - Kafka Topic Configuration: Log Retention
    - Kafka retention—Types, challenges, alternatives

Confluent (Kafka as a Service)

https://docs.confluent.io/kafka/overview.html

Kafka vs RabbitMQ vs SQS

- https://ably.com/topic/apache-kafka-vs-rabbitmq-vs-aws-sns-sqs
- https://aws.amazon.com/compare/the-difference-between-rabbitmq-and-kafka/
- https://www.svix.com/resources/faq/kafka-vs-sqs/
- https://www.svix.com/resources/faq/rabbitmq-vs-sqs/

Zookeeper

- **Design goals:** https://zookeeper.apache.org/doc/current/zookeeperOver.html#sc_designGoals
- **Data Model (Persistent/Ephemeral nodes)**
    - https://zookeeper.apache.org/doc/r3.4.13/zookeeperOver.html#sc_dataModelNameSpace
    - https://zookeeper.apache.org/doc/r3.4.13/zookeeperProgrammers.html#ch_zkDataModel
- **Use Zookeeper to implement**
    - Distributed Priority Queue: https://zookeeper.apache.org/doc/r3.4.13/recipes.html#sc_recipes_Queues

        - Distributed Locks: https://zookeeper.apache.org/doc/r3.4.13/recipes.html#sc_recipes_Locks
        - Two Phased Commit: https://zookeeper.apache.org/doc/r3.4.13/recipes.html#sc_recipes_twoPhasedCommit
        - Master Election in Master Slave: https://zookeeper.apache.org/doc/r3.4.13/recipes.html#sc_leaderElection
- **Code** *(just for a quick glance - don't try to read the code thoroughly)*
    - Observer pattern implementation
        - ObserverBean.java
        - WatchManager.java
    - File structure implementation
        - https://zookeeper.apache.org/doc/r3.9.1/zookeeperOver.html#sc_dataModelNameSpace
        - DataNode.java
        - DataTree.java
    - Leader Election implementation
        - algorithm overview: https://www.quora.com/How-is-a-leader-elected-in-Apache-ZooKeeper
        - FastLeaderElection.java

Doubts

Conflict resolution: use CRDTs